C07 # Team Conventions at Scale

Design Advanced

Synchronizing Claude Code practices across a team of 5+ developers

The N×M×P problem

In a team of 5 developers using 3 AI tools on 2 operating systems, you potentially have 30 different configurations. In practice, after 3 months, no two developers have the same CLAUDE.md: Alice has added TypeScript rules that Bob never received, Carol configured macOS paths that break on Linux.

This is not a discipline problem. It is a configuration architecture problem.

Solution: Profile-Based module assembly

Instead of a shared monolithic file, the recommended structure breaks instructions into reusable modules assembled by profile.

```
profiles/          ← 1 YAML file per developer
  alice.yaml
  bob.yaml
modules/           ← Atomic instructions per topic
  core-standards.md
  typescript-rules.md
  git-workflow.md
  macos-paths.md
skeleton/          ← Template with placeholders
  claude.md
sync-script.ts     ← Automatic assembler
```

An updated module propagates automatically to all developers on the next `sync`.

Developer profile (YAML)

```yaml
# profiles/alice.yaml
name: "Alice"
os: "macos"
tools: [claude-code, cursor]
modules:
  core: [core-standards, git-workflow, typescript-rules]
  conditional: [macos-paths, cursor-rules]
```

Alice receives only the modules she needs. Bob on Linux without Cursor receives a different CLAUDE.md, generated from the same sources.

Measure the result

Compare loaded instruction size, update propagation failures and accepted-task outcomes before and after introducing profiles. No team benchmark is supplied here; the architecture needs validation on your own configurations.

Versioned settings.json = team rules

Shared rules (hooks, allowed permissions) go into the versioned `.claude/settings.json`. Personal overrides go into the gitignored `.claude/settings.local.json`.

```jsonc
// .claude/settings.json (team, in the repository)
{
  "hooks": {
    "PreToolUse": [{ "matcher": "Bash", "hooks": [...] }]
  }
}
```

This separates shared defaults from personal choices. Enforce mandatory restrictions through managed policy and verify the effective configuration.

Break-even point

Use an assembler when repeated configuration drift costs more than maintaining the profiles. Team size alone does not establish a break-even point.

For smaller teams, the simple strategy remains effective: one shared versioned CLAUDE.md, one `settings.local.json` per developer for overrides.